RELATORIO TECNICO

Decodificador de Instrucoes RISC-V

Avaliacao Pratica M1 - Etapa 1 de 2

Organizacao de Computadores | UNIVALI

INTEGRANTES
Jose Gabriel Santos Gomes
Matheus Pompeo Dias
Nathan Gustavo Reichert

Objetivo

Desenvolver um programa em C++17 que le palavras de uma ROM, identifica e caracteriza instrucoes RISC-V de 32 bits, apresenta uma listagem em assembly e calcula estatisticas do trecho analisado. A etapa prioriza a identificacao correta dos campos para que a estrutura possa ser ampliada na etapa seguinte.

Escopo da Etapa 1

O programa analisa cada palavra e produz sua representacao legivel. Ele nao simula os valores dos registradores, a memoria de dados ou a execucao do programa. O codigo mantem metodos que informam registradores lidos/escritos e alvos de desvios para apoiar a extensao futura, mas nao detecta hazards nesta etapa.

Fluxo resumido

ROM texto em hex/bin	**Parser** valor e PC	**Decoder** formato e campos	**Saida** assembly e estatisticas

A instrucao 0x00500413, por exemplo, e apresentada como addi s0, zero, 5: uma soma de 5 com zero cujo destino e s0.

1. Requisitos atendidos pelo projeto

O enunciado organiza a atividade em cinco requisitos. A tabela relaciona cada requisito com os componentes correspondentes da implementacao.

REQ.	RESULTADO ESPERADO	IMPLEMENTACAO NO PROJETO
R1 Entrada	Ler uma instrucao por linha em hexadecimal ou binario, detectar a base, ignorar linhas vazias/comentarios/prefixo 0x e atribuir PC a partir de base configuravel.	parseStream em src/parser.cpp limpa comentarios e espacos, detecta a base, converte palavras e avanca o PC em 4 bytes por palavra aceita. A CLI recebe --base/-b.
R2 Identificacao	Classificar em R/I/S/B/U/J, combinar opcode, funct3 e funct7 e reportar palavras desconhecidas sem interromper o arquivo.	decode em src/decoder.cpp direciona pelo opcode para funcoes especializadas. Combinacoes desconhecidas sao marcadas Invalid e conservam PC, palavra e motivo.
R3 Campos	Extrair somente campos validos no formato e remontar imediatos, inclusive B/J com bits em posicoes diferentes e sinal.	Instruction guarda campos em optional. immediateI/S/B/U/J interpretam cada disposicao; signExtend trata valores negativos. S/B nao criam rd.
R4 Saida	Mostrar endereco, palavra, formato, mnemônico, campos e assembly com nomes ABI. Para B/J, mostrar alvo absoluto e indicar pseudoinstrucoes comuns.	printListing em src/main.cpp monta a listagem; toAssembly, fieldsToString, abiName e toPseudo ficam em src/disassembler.cpp. branchTarget calcula PC + imediato para B/J.
R5 Estatisticas	Contar instrucoes validas por formato, percentuais e CPI medio ponderado usando tabela de custos por classe.	computeStatistics em src/statistics.cpp acumula formato/classe e soma custos. --cpi/-c permite carregar a tabela. A saida estatistica fica em src/main.cpp.

Distribuicao da nota no enunciado

CRITERIO	PONTOS	CRITERIO	PONTOS
R1 - entrada robusta	1,0	R3 - imediatos e sinal	2,0
R2 - seis formatos	1,5	R4 - assembly, ABI e alvos	1,0
R2 - mnemônicos	2,0	R5 - estatisticas e CPI	1,0
R3 - campos por formato	1,5	**Total**	**10,0**

A defesa e individual. O enunciado informa que ela pode ajustar a nota de cada integrante em relacao ao resultado do grupo.

2. Arquitetura e decodificacao

A aplicacao separa leitura, interpretacao, formatacao e estatisticas. O main coordena o fluxo; a biblioteca riscv_core, configurada no CMake, concentra parser, decoder, disassembler e statistics.

ARQUIVO	RESPONSABILIDADE
src/main.cpp	Le argumentos e arquivos; chama as etapas; imprime listagem, avisos e estatisticas.
include/riscv.hpp	Define Format, Class, Instruction, RawWord, ParseResult, CpiTable, Statistics e assinaturas das funcoes.
src/parser.cpp	Transforma linhas da ROM em RawWord{pc, value, line}; reporta erros de conversao.
src/decoder.cpp	Extrai bits, identifica formato/classe/mnemônico, monta imediatos e marca combinacoes invalidas.
src/disassembler.cpp	Converte Instruction em assembly com nomes ABI; calcula texto de campos e pseudoinstrucoes.
src/statistics.cpp	Carrega valores padrao/personalizados e calcula contagens e CPI medio.
tests/test_decoder.cpp	Contem verificacoes unitarias sem framework externo, incluindo exemplos, imediatos, parser e CPI.

Representacao de uma instrucao

Instruction registra PC, palavra original, formato, classe, mnemônico, erro e campos opcionais. Um campo ausente usa std::nullopt; zero e um valor valido. Essa diferenca evita tratar bits de um imediato como se fossem um registrador inexistente. Os metodos readsRegisters e writesRegister omitem x0, cujo valor e fixo.

FORMATO	OPERANDOS TIPICOS	CAMPOS IMPORTANTES
R	add a0, a1, a2	rd, rs1, rs2, funct3, funct7
I	addi a0, a1, 5; lw a0, 0(sp)	rd/rs1 e/ou imediato; o conteudo depende da operacao.
S	sw a0, 8(sp)	rs1, rs2, imediato; sem rd.
B	beq a0, a1, alvo	rs1, rs2, imediato; sem rd.
U	lui a0, 0x12345	rd e imediato; sem rs1, rs2 e funct3.
J	jal ra, alvo	rd e imediato do salto.

Imediatos B e J tem bits espalhados na palavra; as funcoes immediateB e immediateJ remontam esses pedacos antes da extensao de sinal. branchTarget soma o deslocamento assinado ao PC da propria instrucao. O alvo do jalr depende de rs1 em tempo de execucao e nao e calculado pelo analisador.

3. Saida, estatisticas e validacao

A listagem preserva o endereco e a palavra original para facilitar a conferencia. Instrucoes invalidas aparecem com seu motivo e nao interrompem o processamento. Pseudoinstrucoes sao exibidas como uma observacao depois do assembly base.

CPI ponderado

A tabela padrao associa CPI a oito classes: alu=1, muldiv=3, load=2, store=1, branch=2, jump=2, upper=1 e system=1. Uma tabela recebida por --cpi substitui valores das classes especificadas e conserva os demais padroes.

CPI medio = soma(quantidade da classe x CPI da classe) / total de instrucoes validas.

O valor e uma estimativa do modelo de custos informado, nao uma cronometragem de execucao.

Casos de validacao incluidos

tests/test_decoder.cpp cobre exemplos do enunciado (addi, add, sw, beq), todos os formatos, imediatos positivos/negativos, campos ausentes, operacoes como add/sub e srli/srai, pseudoinstrucoes, registradores lidos/escritos, deteccao hex/bin, linhas invalidas e CPI ponderado.

tests/data/programa.hex e tests/data/programa.bin contem a mesma sequencia de exemplo: ao menos uma instrucao R/I/S/B/U/J, um desvio com deslocamento negativo, salto negativo e palavra invalida. tests/data/cpi.txt mostra a tabela por classe. O projeto inclui os testes e as entradas para reproducao; este relatorio nao afirma que uma execucao dos testes tenha sido realizada nesta montagem.

Compilacao e execucao

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
./build/riscv-decoder tests/data/programa.hex --cpi tests/data/cpi.txt
./build/riscv-decoder tests/data/programa.bin --base 0x00400000
```

O projeto requer CMake 3.16 ou superior e compilador com C++17. O CMake cria a biblioteca riscv_core, o executavel riscv-decoder e o alvo de testes test_decoder.

Conteudo do pacote

O ZIP inclui CMakeLists.txt, README.md, include/riscv.hpp, os fontes em src/, test_decoder.cpp, as duas ROMs de exemplo e a tabela de CPI. Inclui este relatorio. A pasta .git, o PDF do enunciado e artefatos de compilacao nao sao necessarios para construir o programa.

Referencias

Enunciado M1: github.com/mapompeo/riscv-decoder/blob/main/riscv-decoder.pdf
Codigo-fonte: github.com/mapompeo/riscv-decoder